

JDBC_Source

Warning

En este lab se supone instalada la base de datos *Sakila* en un contenedor accesible para el *ksqldb-server*

Existen diversas formas de conectar con una base de datos en un contenedor *Docker*. Se recomienda exportar un puerto local no ocupado, por ejemplo el 33060. De esta forma, la conexión es totalmente análoga a como sería conectar con un gestor instalado de forma nativa en la máquina local:

```
-- En lo que sigue se supone creado un usuario santiago con permisos sobre sakila
mysql -h 127.0.0.1 -u santiago -P 33060 -p
describe sakila.customer;
select * from sakila.customer;
```

A continuación se muestran varias definiciones de conectores de la clase *JdbcSourceConnector*. Ojo con los valores de las propiedades *user* y *password* que debieran ser las que, en cada caso, procedan

Conexión API REST cruda (raw) a una tabla simple (customer)

Crear JdbcSourceConnector (raw)

```
curl -X POST http://localhost:8083/connectors -H "Content-Type: application/json" -d '{
  "name": "raw_customer",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "tasks.max": "1",
    "connection.url": "jdbc:mysql://mariadb:3306/sakila?serverTimezone=Europe/Madrid",
    "connection.user": "sak_user",
    "connection.password": "sak_pass",
    "table.whitelist": "customer",
    "mode": "incrementing",
    "incrementing.column.name": "customer_id",
    "validate.non.null": "false",
    "topic.prefix": "raw."
  }
}
```

```
}  
}'
```

Conexión ksql básica

Crear JdbcSourceConnector (raw)

```
CREATE SOURCE CONNECTOR `raw_customer` WITH(  
  "connector.class" = 'io.confluent.connect.jdbc.JdbcSourceConnector',  
  "connection.url" = 'jdbc:mysql://mariadb:3306/sakila?  
serverTimezone=Europe/Madrid',  
  "connection.user" = 'sak_user',  
  "connection.password" = 'sak_pass',  
  "db.timezone" = 'Europe/Madrid',  
  "table.whitelist" = 'customer',  
  "mode" = 'incrementing',  
  "incrementing.column.name" = 'customer_id',  
  "validate.non.null" = 'false',  
  "topic.prefix" = 'raw.',  
  "tasks.max" = '1');  
  
print 'raw.customer' from beginning limit 5;
```

Crear Kafka stream (raw)

```
create stream `raw_customer` (  
  customer_id INTEGER,  
  store_id INTEGER,  
  first_name STRING,  
  last_name STRING,  
  email STRING,  
  last_update BIGINT  
)  
WITH (  
  KAFKA_TOPIC='raw.customer',  
  VALUE_FORMAT='AVRO'  
);
```

Conexión ksql con transformación de clave

Crear JdbcSourceConnector (**OJO**: key default type JSON or KAFKA_STRING)

```
CREATE SOURCE CONNECTOR `sak_customer` WITH(  
  "connector.class" = 'io.confluent.connect.jdbc.JdbcSourceConnector',  
  "connection.url" = 'jdbc:mysql://mariadb:3306/sakila?  
serverTimezone=Europe/Madrid',  
  "connection.user" = 'sak_user',  
  "connection.password" = 'sak_pass',
```

```

        "table.whitelist" = 'customer',
        "mode" = 'incrementing',
        "incrementing.column.name" = 'customer_id',
        "validate.non.null" = 'false',
        "topic.prefix" = 'sak.',
        "transforms" = 'createKey,extractInt',
        "transforms.createKey.type" =
'org.apache.kafka.connect.transforms.ValueToKey',
        "transforms.createKey.fields" = 'customer_id',
        "transforms.extractInt.type" =
'org.apache.kafka.connect.transforms.ExtractField$Key',
        "transforms.extractInt.field" = 'customer_id',
        "tasks.max" = '1');

```

Crear Kafka stream

```

create stream `sak_customer` (
  customer_id INTEGER,
  store_id INTEGER,
  first_name STRING,
  last_name STRING,
  email STRING,
  last_update BIGINT)
WITH (kafka_topic='sak.customer', value_format='AVRO',
TIMESTAMP='last_update');

```

Crear JdbcSourceConnector (key/value schema)

En este caso se fuerza la **conversión de la Key del mensaje a _KAFKA_INT**

```

CREATE SOURCE CONNECTOR `sak_customer_sch` WITH(
  "connector.class" = 'io.confluent.connect.jdbc.JdbcSourceConnector',
  "connection.url" = 'jdbc:mysql://mariadb:3306/sakila?
serverTimezone=Europe/Madrid',
  "connection.user" = 'sak_user',
  "connection.password" = 'sak_pass',
  "key.converter" = 'org.apache.kafka.connect.converters.IntegerConverter',
  "key.converter.schema.registry.url" = 'http://schema-registry:8081',
  "key.converter.schemas.enable" = 'true',
  "value.converter" = 'io.confluent.connect.avro.AvroConverter',
  "value.converter.schema.registry.url" = 'http://schema-registry:8081',
  "value.converter.schemas.enable" = 'true',
  "table.whitelist" = 'customer',
  "mode" = 'incrementing',
  "incrementing.column.name" = 'customer_id',
  "validate.non.null" = 'false',
  "topic.prefix" = 'sch.',
  "transforms" = 'createKey,extractInt',
  "transforms.createKey.type" =
'org.apache.kafka.connect.transforms.ValueToKey',
  "transforms.createKey.fields" = 'customer_id',
  "transforms.extractInt.type" =

```

```
'org.apache.kafka.connect.transforms.ExtractField$Key',  
  "transforms.extractInt.field" = 'customer_id',  
  "tasks.max" = '1');
```

Comprobaciones

```
print 'sak.customer' from beginning limit 2;  
print 'sch.customer' from beginning limit 2;  
  
show streams;  
describe `raw_customer`;  
describe extended `raw_customer`;  
describe `sak_customer`;  
describe extended `sak_customer`;  
  
SET 'auto.offset.reset'='earliest';  
  
select first_name, last_name, email from `s_customer` emit changes;
```

Ejemplo de consulta con agregados

```
select store_id, count(*) as n from `sak_customer` group by store_id emit  
changes;  
  
CREATE TABLE c_store AS  
  SELECT store_id, COUNT(*)  
  FROM `sak_customer`  
  GROUP BY store_id  
  EMIT CHANGES;  
-- Pull Query  
select * from c_store where store_id=1;
```